

Linear Regression

1. The Intuition: Fitting a Line

Linear Regression is the "Hello World" of Machine Learning. It's about finding the best straight line to fit your data.

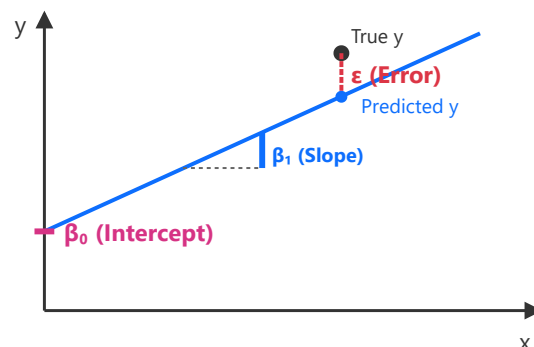
Imagine you want to predict house prices based on size:

- **X (Input):** Size of the house (sq ft).
- **Y (Output):** Price (\$).

We assume there is a linear relationship:

$$y = \beta_0 + \beta_1 x + \epsilon$$

- y Target:** What we want to predict (e.g., Price).
- β_0 Intercept:** The starting value when x is 0.
- β_1 Slope (Weight):** How much y changes for each unit of x.
- ϵ Error (Residual):** The noise or "miss" distance from the line.



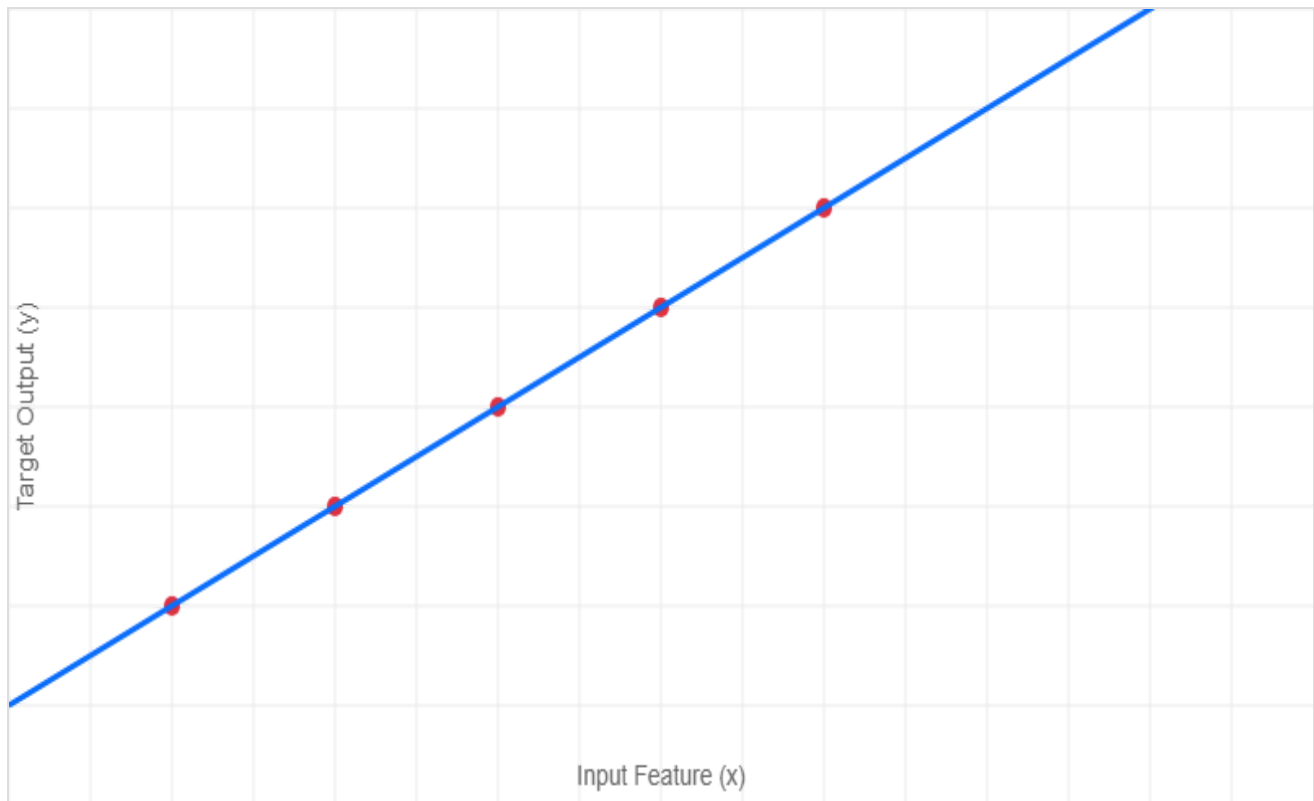
Or in vector notation for multiple features:

$$y = \beta^\top x + \epsilon$$

Our goal is to find the best numbers for β (weights) so that our line passes as close as possible to all the real data points.

2. Interactive Visualization

Click on the canvas below to add data points. The blue line will automatically update to show the "Best Fit" (Least Squares) regression line.



Learned Model: $y = -0.50x + 350.00$

3. The Math: Least Squares

We define the "best" line as the one that minimizes the sum of squared errors (SSE).

The Objective Function

$$\hat{\beta} = \arg \min_{\beta, \beta_0} \sum_{i=1}^n (y_i - (\beta^\top x_i + \beta_0))^2$$

We want to find parameters β (slope) and β_0 (intercept) that minimize the squared difference between the true value y_i and our prediction.

The Solution (Normal Equation)

$$\hat{\beta} = (X^\top X)^{-1} X^\top y$$

This is the closed-form solution using matrix algebra. It gives us the optimal coefficients directly in one step!

Tiny Dataset Example (2 Samples, 2 Features):

Matrix X (Inputs):

Sample 1: [1, 2] (Bias=1, Feat=2)

Sample 2: [1, 4] (Bias=1, Feat=4)

$$X = \begin{bmatrix} 1 & 2 \\ 1 & 4 \end{bmatrix}$$

Matrix $X^\top$ (Transpose):

Swap rows and columns

$$X^\top = \begin{bmatrix} 1 & 1 \\ 2 & 4 \end{bmatrix}$$

Vector y (Targets):

Target 1: 5

Target 2: 9

$$y = \begin{bmatrix} 5 \\ 9 \end{bmatrix}$$

The Magic Calculation:

$$(X^\top X)^{-1} X^\top y = \begin{bmatrix} 1 \\ 2 \end{bmatrix} \Rightarrow \text{Bias} = 1, \text{Slope} = 2$$

Prediction check: $1 + 2(2) = 5$, $1 + 2(4) = 9$. Perfect match!

4. Python Implementation

Here is how you implement Linear Regression using [scikit-learn](#). Notice how simple the API is!

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.datasets import load_diabetes
import matplotlib.pyplot as plt
import numpy as np

def regression_demo_linear():
    """
    Regression demo on diabetes dataset using Linear Regression.
    """
    # 1. Load Data
    # For visualization purposes, we'll use only one feature (BMI)
    X, y = load_diabetes(return_X_y=True)
    print(f"Shape of X: {X.shape}")
    print(f"Shape of y: {y.shape}")
    X = X[:, np.newaxis, 2] # Use only BMI feature

    X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=1)

    # 2. Train Model
    model = LinearRegression()
    model.fit(X_train, y_train) # Train

    # 3. Evaluate
    y_pred = model.predict(X_test) # Predict

    mse = mean_squared_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)
    print(f"LinearRegression: MSE={mse:.2f}, R2={r2:.3f}")

    # 4. Visualize
```

```
plt.scatter(X_test, y_test, color='black', label='Actual Data')
plt.plot(X_test, y_pred, color='blue', linewidth=3, label='Learned
Line')
plt.xlabel('BMI (Normalized)')
plt.ylabel('Disease Progression')
plt.title('Linear Regression Model')
plt.legend()
plt.show()

# calling the function
regression_demo_linear()
```